

Chương này trình bày thiết kế và quá trình triển khai WoodCert Auction, từ kiến trúc tổng thể, giao diện, lớp và cơ sở dữ liệu đến kết quả xây dựng, kiểm thử và vận hành trên VPS. Nội dung tập trung vào cách các thành phần phối hợp trong những luồng nghiệp vụ chính, đồng thời nêu rõ các giới hạn kỹ thuật còn tồn tại.

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

WoodCert Auction sử dụng kiến trúc đơn khối phân mô-đun (modular monolith) cho phía máy chủ và kiến trúc ứng dụng trang đơn (Single Page Application – SPA) cho phía giao diện. Hai khái niệm này mô tả hai cấp khác nhau: modular monolith xác định cách tổ chức và triển khai backend, còn SPA xác định cách frontend được tải và thực thi trong trình duyệt. Backend được đóng gói thành một ứng dụng Spring Boot chạy trong một tiến trình Java, sử dụng một schema MySQL và kết nối Redis. Frontend được biên dịch thành các tệp tĩnh và phân phát qua Nginx. Vì vậy, thuật ngữ đơn khối trong báo cáo chỉ áp dụng cho backend, không có nghĩa toàn bộ hệ thống cùng chạy trong một tiến trình.

Quyết định lựa chọn modular monolith xuất phát từ ba đặc điểm của bài toán. Thứ nhất, các nghiệp vụ đấu giá, ví, đơn hàng, giao nhận và tranh chấp phải phối hợp nhiều trạng thái trên cùng dữ liệu quan hệ. Việc đặt các mô-đun trong cùng ứng dụng cho phép gọi hàm nội bộ và xác định giao dịch MySQL cục bộ cho từng bước nghiệp vụ, thay vì phải xử lý lỗi mạng giữa các dịch vụ. Điều này không có nghĩa toàn bộ chuỗi từ đặt giá đến giao hàng nằm trong một giao dịch duy nhất; Redis, WebSocket và dịch vụ ngoài không cùng tham gia giao dịch ACID của MySQL. Thứ hai, phạm vi đồ án phù hợp với nhóm phát triển nhỏ và môi trường triển khai VPS; tách thành nhiều dịch vụ độc lập sẽ làm tăng cấu hình mạng, yêu cầu quan sát hệ thống, quy trình triển khai và các tình huống lỗi phân tán. Thứ ba, các miền nghiệp vụ vẫn cần được phân chia rõ để tránh tập trung toàn bộ logic trong một số lớp lớn và tạo điều kiện bảo trì.

Bảng ?? so sánh ba phương án kiến trúc được xem xét. So với đơn khối phân lớp không chia miền, modular monolith yêu cầu kỷ luật cao hơn trong tổ chức package và giao tiếp liên mô-đun, nhưng giúp trách nhiệm nghiệp vụ rõ ràng hơn. So với vi dịch vụ, phương án hiện tại không hỗ trợ triển khai và mở rộng tài nguyên độc lập cho từng miền, đổi lại hệ thống tránh giao dịch phân tán và giảm chi phí vận hành.

Tiêu chí	Đơn khối phân lớp	Đơn khối phân mô-đun	Vi dịch vụ
Ranh giới nghiệp vụ	Dễ bị trộn giữa các tầng dùng chung	Tách theo miền trong cùng ứng dụng	Tách theo dịch vụ và tiến trình
Giao dịch dữ liệu quan hệ	Dễ dùng giao dịch cục bộ nhưng ranh giới nghiệp vụ thường mờ	Có thể đặt giao dịch MySQL theo từng use case liên mô-đun	Giao dịch xuyên dịch vụ cần thiết kể riêng hoặc dùng nhất quán cuối
Triển khai	Đơn giản	Một backend chính, phù hợp VPS hiện tại	Phức tạp hơn do nhiều đơn vị triển khai
Mở rộng độc lập	Hạn chế	Hạn chế	Thuận lợi hơn
Mức phù hợp với đồ án	Có thể khó bảo trì khi nghiệp vụ tăng	Phù hợp phạm vi, nhóm phát triển và hạ tầng	Chi phí vận hành vượt nhu cầu hiện tại

Bảng 1: So sánh các phương án kiến trúc phần mềm

Backend được chia thành vùng *core* và vùng *feature*. Vùng *core* chứa cấu trúc phản hồi API, xử lý ngoại lệ, bảo mật, JWT, WebSocket, thời gian hệ thống và các thành phần dùng chung. Vùng *feature* gồm tám mô-đun nghiệp vụ: *identity*, *media*, *catalog*, *finance*, *auction*, *order*, *fulfillment* và *dispute*. Cách tổ chức này tạo ranh giới theo miền nghiệp vụ, nhưng chưa đạt mức cô lập hoàn toàn. Một số điểm giao tiếp đã dùng interface hoặc port/adapter, trong khi một số mô-đun vẫn phụ thuộc trực tiếp vào nhau.

Trong từng mô-đun, mã nguồn tiếp tục được tổ chức theo kiến trúc phân lớp gồm controller, DTO, service, repository và entity. Controller tiếp nhận yêu cầu HTTP và kiểm tra đầu vào; service thực hiện quy tắc nghiệp vụ; repository truy cập MySQL qua Spring Data JPA; entity ánh xạ dữ liệu bền vững; DTO xác định hợp đồng dữ liệu tại biên API. Với miền phức tạp, tầng service được tách tiếp theo trách nhiệm. Chẳng hạn, mô-đun *auction* có service riêng cho command, query, vòng đời phiên, đặt giá, quyết toán cọc và phát sự kiện thời gian thực.

Lợi ích chính của lựa chọn này là khả năng phối hợp nghiệp vụ mà không phát sinh giao tiếp mạng giữa các mô-đun nội bộ. Các bước tạo phiên, đóng băng cọc, chốt trạng thái phiên và tạo đơn có thể đặt biên giao dịch Spring phù hợp với phần dữ liệu MySQL mà mỗi bước quản lý. Việc triển khai cũng tập trung vào một image backend, một image frontend và các dịch vụ lưu trữ.

Đánh đổi của kiến trúc là sự cố tài nguyên hoặc lỗi nghiêm trọng trong một phần backend có thể ảnh hưởng đến toàn bộ ứng dụng. Ranh giới package hiện chủ yếu được duy trì bằng quy ước phát triển; dự án chưa dùng Spring Modulith, ArchUnit hoặc công cụ tương đương để kiểm soát tự động chiều phụ thuộc. Một số package vẫn truy cập trực tiếp entity hoặc repository của package khác. Vì vậy, các mô-đun có trách nhiệm tương đối rõ nhưng chưa phải những đơn vị độc lập.

Điểm đặc thù của WoodCert Auction nằm ở cách phân chia trách nhiệm trạng thái giữa MySQL và Redis. MySQL quản lý dữ liệu bền vững như cấu hình phiên, người tham gia, lịch sử bid đã ghi thành công, đơn hàng và trạng thái kết thúc. Khi phiên ở trạng thái `ACTIVE`, Redis giữ giá hiện tại, người dẫn đầu, thời điểm kết thúc và tập người đủ điều kiện đặt giá. Lua script kiểm tra và cập nhật nguyên tử; mã `OK` là ranh giới chấp nhận một lượt bid. Sau đó backend phát sự kiện và gọi tuần tự hai bước ghi MySQL theo cơ chế best-effort. Hai bước này không cùng giao dịch ACID với Redis và lỗi không làm đảo ngược bid đã được chấp nhận.

Giới hạn còn lại xuất hiện khi việc lưu bản ghi bid và đồng bộ snapshot MySQL đều thất bại, đồng thời state Redis không còn khả dụng lúc đóng phiên. Fallback từ MySQL khi đó có thể cũ hơn trạng thái Redis cuối cùng. Nếu state Redis vẫn tồn tại, worker đóng phiên đọc trực tiếp trạng thái đó. Đây là sự đánh đổi có chủ đích để ưu tiên đường xử lý thời gian thực, không phải bảo đảm tuyệt đối không mất dữ liệu.

0.1.2 Thiết kế tổng quan

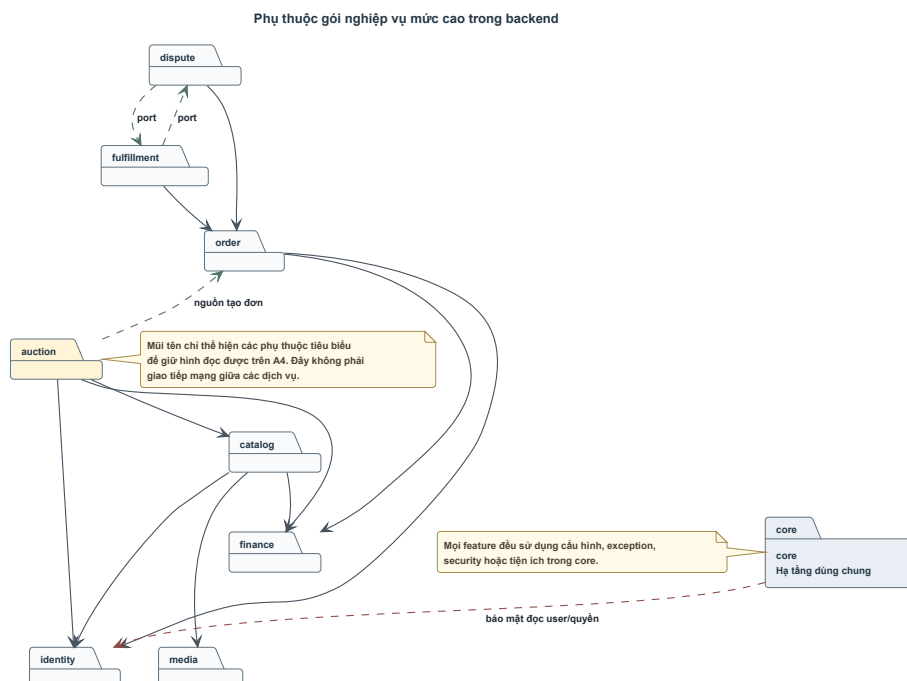
Ở mức tổng quan, hệ thống gồm trình duyệt người dùng, frontend React, backend Spring Boot, MySQL, Redis, Nginx và ba tích hợp ngoài là Cloudinary, VNPay Sandbox và SMTP. Trình duyệt tải SPA từ Nginx, gửi yêu cầu REST tới backend và mở kết nối STOMP/WebSocket cho phòng đấu giá. Backend xử lý nghiệp vụ, lưu dữ liệu bền vững trong MySQL và sử dụng Redis cho trạng thái runtime, khóa tạm thời và một số bộ đếm bảo mật. Đối với tệp đa phương tiện, frontend tải trực tiếp lên Cloudinary bằng thông tin upload có chữ ký ngắn hạn do backend cấp; backend xác nhận lại tài sản trước khi cho phép liên kết với dữ liệu nghiệp vụ.

Hình ?? trình bày các phụ thuộc chính giữa các package backend. `core` cung cấp phần lớn thành phần dùng chung cho `feature`, nhưng một số lớp bảo mật trong `core` vẫn sử dụng entity và repository của `identity`. Trong vùng nghiệp vụ, `catalog` dùng dữ liệu từ `identity`, `media` và `finance`; `auction` phối hợp với `catalog`, `identity`, `finance` và chuyển nguồn tạo đơn sang `order`; `order` tiếp tục làm việc với `finance`, `identity` và `fulfillment`.

Một số biên giao tiếp đã được tách qua interface hoặc adapter. Adapter nguồn

đơn hàng giúp order lấy snapshot từ phiên đấu giá. Các cổng của fulfillment hỗ trợ order và dispute phối hợp trạng thái giao nhận; các dịch vụ snapshot trong identity cung cấp thông tin người mua, người bán và địa chỉ giao hàng. Tuy vậy, phụ thuộc trực tiếp giữa một số package vẫn còn tồn tại.

Frontend cũng được tổ chức theo feature tương ứng với các khu vực auth, account, seller, appraisal, auction, bidding, wallet, buyer, order, fulfillment, dispute và admin. Các route public dùng PublicLayout; seller, appraiser và admin có layout cùng guard riêng; phòng đặt giá sử dụng layout toàn màn hình. TanStack Query giữ server state và cache truy vấn, Zustand giữ access token trong bộ nhớ, còn API client chịu trách nhiệm gắn token, làm mới phiên bằng refresh cookie và thử lại yêu cầu sau lỗi xác thực.



Hình 0.1: Phụ thuộc gói nghiệp vụ mức cao trong backend

Nguồn: Tổng hợp từ quan hệ phụ thuộc trong backend.

0.1.3 Thiết kế chi tiết gói

Mô-đun auction được chọn để minh họa thiết kế package vì đây là nơi phối hợp REST API, Redis, MySQL, WebSocket, scheduler và các mô-đun tài chính, sản phẩm, đơn hàng. Thiết kế không tập trung vào một service duy nhất mà phân thành nhóm quản lý phiên, nhóm đặt giá runtime, nhóm vòng đời và nhóm quyết toán. Cấu trúc rút gọn được thể hiện tại Hình ??.

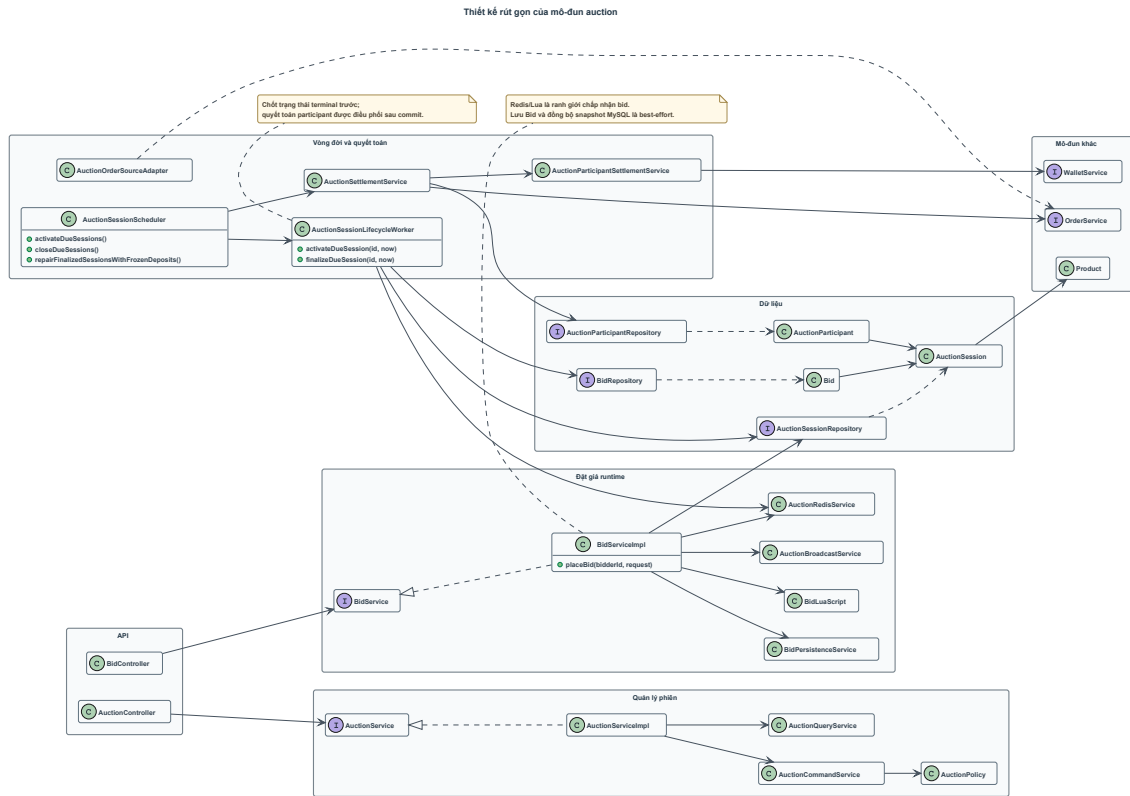
AuctionController tiếp nhận các yêu cầu xem phiên, tạo phiên, đăng ký, rút đăng ký, hủy phiên và xem dữ liệu seller/buyer. Controller gọi AuctionSer-

vice; AuctionServiceImpl đóng vai trò facade và chuyển tiếp thao tác ghi tới AuctionCommandService, thao tác đọc tới AuctionQueryService hoặc BuyerAuctionQueryService. AuctionPolicy tập trung giới hạn về trạng thái sản phẩm, thời gian bắt đầu, thời lượng, bước giá, giá bảo lưu và tiền cọc.

Đường đặt giá có entry point riêng là BidController. BidServiceImpl kiểm tra sơ bộ trạng thái phiên cùng quyền sở hữu sản phẩm từ MySQL, sau đó chạy BidLuaScript trên Redis. Nếu script chấp nhận, AuctionBroadcastService phát sự kiện NEW_BID; BidPersistenceService lưu audit bid trong transaction độc lập; repository cập nhật snapshot phiên theo điều kiện chống ghi đè dữ liệu mới bằng dữ liệu cũ. Hai bước ghi MySQL được gọi tuần tự trong request nhưng mang tính best-effort.

Vòng đời phiên được điều khiển bởi AuctionSessionScheduler. Scheduler kích hoạt phiên đến hạn, đóng phiên hết hạn và sửa chữa những phiên terminal còn participant chưa quyết toán hoặc thiếu đơn hàng. Lifecycle worker khóa phiên, đọc snapshot Redis hoặc fallback MySQL, xác định phiên kết thúc thành công hay thất bại, rồi cập nhật trạng thái bền vững và trạng thái bán của sản phẩm. Sau khi trạng thái terminal được commit, settlement service điều phối quyết toán. Mỗi participant được xử lý trong transaction REQUIRES_NEW, giúp lỗi của một participant không rollback toàn bộ danh sách.

Các entity chính gồm AuctionSession, AuctionParticipant và Bid. AuctionSession lưu cấu hình cùng snapshot MySQL; AuctionParticipant lưu trạng thái cọc từ FROZEN sang WITHDRAWN, REFUNDED, DEDUCTED hoặc CONFISCATED; Bid là nhật ký không được cập nhật hay xóa sau khi tạo. Thanh toán phần còn lại, giao hàng, hoa hồng, hoàn tiền và tranh chấp thuộc các mô-đun order, fulfillment, finance và dispute, không được lưu như trạng thái nội bộ của phiên đấu giá.



Hình 0.2: Thiết kế rút gọn của mô-đun đấu giá
 Nguồn: Tổng hợp từ các lớp trong mô-đun auction.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Giao diện WoodCert Auction được xây dựng dưới dạng SPA bằng React và tổ chức theo khu vực sử dụng. Các trang công khai phục vụ khách chưa đăng nhập và buyer; seller, appraiser và admin sử dụng portal có layout riêng; phòng đặt giá dùng bố cục toàn màn hình để ưu tiên giá hiện tại, đồng hồ và thao tác đặt giá. Cách phân chia này giúp mỗi actor tiếp cận đúng nhóm nhiệm vụ mà không phải hiển thị toàn bộ chức năng trong một cấu trúc điều hướng duy nhất.

Thiết kế sử dụng hai ngữ cảnh thị giác chính. Khu vực public và phòng đấu giá dùng nền tối, chữ sáng cùng màu nhấn vàng hổ phách để làm nổi bật giá và hành động chính. Khu vực seller, buyer và các màn hình nghiệp vụ dùng nhiều bề mặt sáng, màu xanh mực cho tiêu đề và nút điều hướng, đồng thời giữ màu trạng thái nhất quán. Các token chính được khai báo tập trung trong `src/index.css`. Bảng ?? trình bày các quy ước chính.

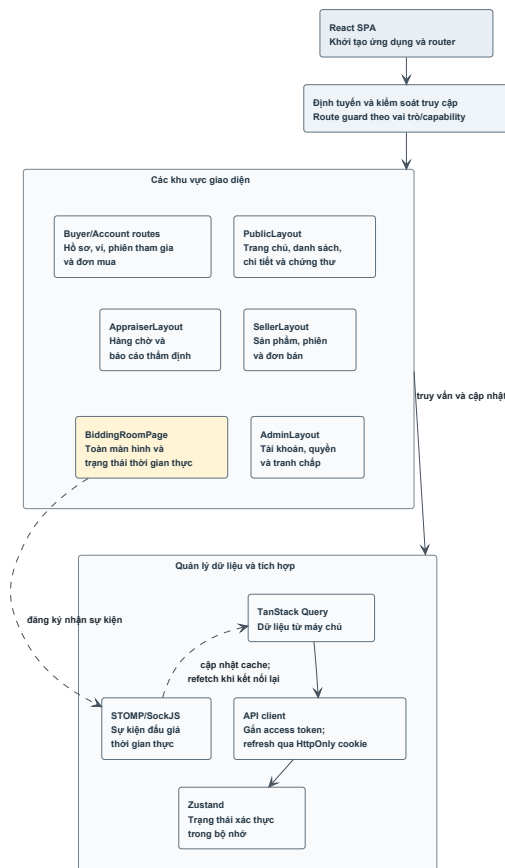
Nhóm token	Giá trị chính trong mã nguồn	Vai trò
Kiểu chữ	Inter; Playfair Display	Inter dùng cho nội dung và portal; Playfair Display tạo phân cấp cho tiêu đề public
Nền thương hiệu	warm-ivory #f6f0e6; nền tối theo OKLCH	Phân biệt portal sáng với khu vực public tối
Màu chính	ink-blue #2e4a62; token vàng hồ phách primary	Điều hướng, hành động chính và giá đấu
Màu trạng thái	verdigris, terracotta, destructive red	Thành công, cảnh báo, lỗi và trạng thái nghiệp vụ
Bán kính	Token cơ sở 0.5rem	Chuẩn hóa input, button, dialog và panel
Chiều rộng nội dung	Nhiều portal dùng max-w-[1280px]	Giới hạn chiều rộng đọc trên màn hình lớn

Bảng 2: Các quy ước thiết kế giao diện chính

Về bố cục, `PublicLayout` cung cấp header, nội dung và footer cho trang chủ, danh sách đấu giá, chi tiết phiên, chứng thư và các trang tài khoản buyer. Seller, appraiser và admin dùng layout riêng có sidebar. `BiddingRoomPage` tách khỏi các layout này và sử dụng giao diện kiểu cockpit gồm trạng thái kết nối, giá trực tiếp, lịch sử bid, thông tin sản phẩm và bảng điều khiển đăng ký hoặc đặt giá. Tổ chức các khu vực giao diện được minh họa tại Hình ??.

Frontend thay đổi bố cục theo các breakpoint của Tailwind CSS. Các khối nhiều cột chuyển thành một cột trên màn hình hẹp; bảng có vùng cuộn ngang hoặc chiều rộng tối thiểu khi dữ liệu không thể thu gọn; các portal ưu tiên thao tác chính và giảm bớt chi tiết thứ cấp. Trang tranh chấp đã được kiểm tra E2E ở kích thước desktop và mobile, còn những màn hình khác được kiểm tra trong phạm vi bộ test hiện có.

Các thành phần dùng chung được xây dựng từ React, Tailwind CSS, `class-variance-authority` và một số primitive của Radix UI. `Button` có các biến thể default, destructive, outline, secondary, ghost và link; mỗi biến thể có trạng thái hover, active, focus-visible và disabled. Dialog dùng Radix Dialog để quản lý focus và lớp phủ; menu tài khoản dùng Radix Dropdown Menu. Radix UI chỉ cung cấp primitive cần thiết, không phải bộ giao diện hoàn chỉnh.



Hình 0.3: Tổ chức các khu vực giao diện

Nguồn: Tổng hợp từ route, layout và cơ chế quản lý trạng thái của frontend.

Form nghiệp vụ thường kết hợp React Hook Form và Zod. Schema phía client kiểm tra email, mật khẩu, số tiền, thời gian phiên, dữ liệu sản phẩm và báo cáo thẩm định trước khi gửi request. Các quy tắc quan trọng vẫn được backend kiểm tra lại; validation phía client chỉ giúp phản hồi sớm, không phải biên bảo mật. Tương tác đặt giá sử dụng điều kiện động từ API gồm `canRegister`, `canWithdraw`, `canBid`, trạng thái cọc và người đang dẫn đầu.

Bảng ?? tổng hợp cách giao diện phản hồi các trạng thái phổ biến. Các mutation vô hiệu hóa nút trong lúc xử lý để hạn chế gửi lặp từ giao diện, trong khi backend tiếp tục dùng operation key và ràng buộc dữ liệu để bảo đảm tính lũy đẳng ở mức nghiệp vụ.

Trạng thái	Cách thể hiện	Nguyên tắc xử lý
Loading	Skeleton, spinner hoặc nút đang xử lý	Giữ bố cục ổn định và ngăn gửi lặp thao tác đang chạy
Empty	Thông báo không có dữ liệu kèm hành động phù hợp	Phân biệt chưa có dữ liệu với lỗi tải dữ liệu
Validation error	Thông báo sát trường nhập	Không dùng validation client làm biên bảo mật
Forbidden hoặc suspended	Trang hoặc chế độ chỉ đọc kèm giải thích	Backend vẫn quyết định quyền hiệu lực
Disconnected	Banner cảnh báo kết nối thời gian thực	Khi kết nối lại, refetch dữ liệu từ API
Success	Toast hoặc cập nhật trạng thái tại chỗ	Vô hiệu hóa thao tác không còn hợp lệ sau mutation

Bảng 3: Ma trận trạng thái phản hồi của giao diện

Đối với phòng đấu giá, frontend đồng bộ đồng hồ với endpoint thời gian server để giảm sai lệch do đồng hồ máy khách. Khi nhận `NEW_BID`, cache chi tiết phiên, lịch sử bid và trạng thái người tham gia được cập nhật. Khi WebSocket kết nối lại, các query liên quan được refetch. REST API và backend vẫn là nguồn xác minh; frontend không tự quyết định lượt bid hợp lệ.

Các route guard dùng claim trong access token để điều hướng người dùng tới portal phù hợp. Đây chỉ là lớp trải nghiệm; backend tải quyền hiệu lực từ cơ sở dữ liệu và kiểm tra trạng thái tài khoản trên mỗi request JWT. Seller portal còn polling hồ sơ định kỳ để chuyển sang chế độ chỉ đọc khi capability `SELLER` bị đình chỉ, nhưng không thay thế kiểm tra `@PreAuthorize` tại API.

0.2.2 Thiết kế lớp

Phần này chọn bốn lớp điều phối tiêu biểu cho các nghiệp vụ đặt giá thời gian thực, quản lý vòng đời phiên, ghi nhận biến động tài chính lũy đẳng và xử lý tranh chấp. Quan hệ giữa các lớp được trình bày tại Hình ??.

a, Lớp `BidServiceImpl`

`BidServiceImpl` triển khai use case đặt giá. Các phụ thuộc của lớp được chia thành bốn nhóm: repository phiên, Lua và Redis, dịch vụ phát sự kiện, cùng thành phần lưu bid và cấu hình đấu giá. Phương thức `placeBid()` kiểm tra phiên tồn tại, trạng thái DB là `ACTIVE` và bidder không phải seller; tạo `bidTraceId`; truyền dữ liệu vào Lua script; sau đó ánh xạ mã kết quả Redis thành phản hồi hoặc lỗi nghiệp vụ.

Khi Lua trả OK, service cập nhật TTL nếu có anti-sniper, phát `NEW_BID`, gọi

lưu audit bid và cập nhật snapshot MySQL. Hai bước MySQL được bọc xử lý lỗi độc lập; lỗi được ghi log nhưng không đảo ngược kết quả đã chấp nhận. Khi script trả LOW, ENDED, NOT_REGISTERED hoặc SELF_BID, service chuyển kết quả thành mã lỗi API tương ứng.

b, Lớp AuctionSessionLifecycleWorker

AuctionSessionLifecycleWorker thực hiện hai thao tác transaction quan trọng là `activateDueSession()` và `finalizeDueSession()`. Khi kích hoạt, lớp khóa phiên WAITING, lấy participant có cọc FROZEN, tải state cùng bidder set vào Redis, sau đó mới chuyển trạng thái MySQL sang ACTIVE. Nếu ghi Redis thất bại, state tạm được dọn và transaction không commit trạng thái active.

Khi đóng phiên, lớp khóa phiên ACTIVE đến hạn và đọc state Redis. Nếu end time đã được anti-sniper kéo dài, lớp đồng bộ snapshot về MySQL và chưa đóng phiên. Nếu state Redis không còn, lớp fallback sang snapshot DB và bid hợp lệ cao nhất. Sau khi xác định giá cuối, người dẫn đầu và việc đạt reserve price, lớp commit ENDED_SUCCESS hoặc ENDED_FAILED, cập nhật trạng thái bán của sản phẩm và trả `CloseResult` cho bước quyết toán.

c, Lớp WalletServiceImpl

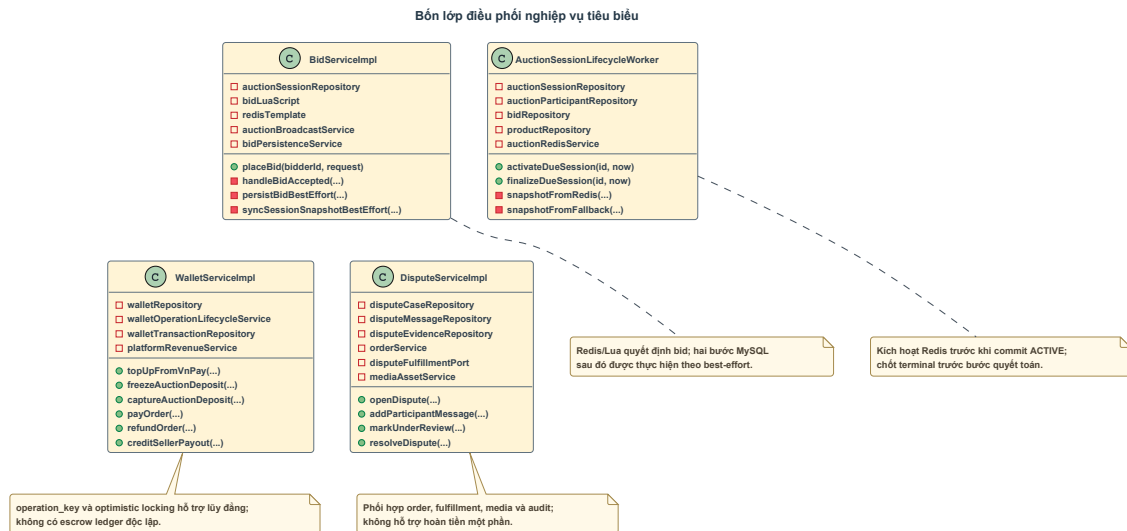
WalletServiceImpl cung cấp biến động số dư cho nạp tiền, phí thẩm định, cọc đấu giá, thanh toán đơn hàng, hoàn tiền, payout seller và bồi thường từ cọc bị tịch thu. Mỗi lệnh nhận một `FinanceOperationKey` xác định nghiệp vụ và tham chiếu. Service chuẩn hóa số tiền, kiểm soát đồng thời, ghi bảng operation để quản lý vòng đời thao tác và tạo bản ghi transaction cho biến động thành công.

Nhóm phương thức chính xử lý nạp tiền, thu phí thẩm định, đóng băng hoặc giải phóng cọc, thu cọc, thanh toán order, hoàn tiền và ghi nhận payout cho seller. Số dư chỉ gồm `availableBalance` và `frozenBalance`; hệ thống không triển khai một sổ cái escrow độc lập. Tiền thanh toán được trừ khỏi ví buyer khi trả phần còn lại và chỉ được cộng vào ví seller khi đủ điều kiện payout; trạng thái trung gian được thể hiện bằng order và lịch sử giao dịch, không phải tài khoản escrow riêng.

d, Lớp DisputeServiceImpl

DisputeServiceImpl quản lý vòng đời vụ việc từ khi buyer mở tranh chấp tới khi admin giải quyết. Service phối hợp repository vụ việc, tin nhắn, bằng chứng, OrderService, cổng fulfillment, media service và audit quản trị. `openDispute()` chỉ chấp nhận khi order và fulfillment đủ điều kiện, yêu cầu ít nhất một bằng chứng mở đầu và ngăn tồn tại đồng thời nhiều dispute OPEN hoặc UNDER_REVIEW cho cùng order.

Buyer có thể `cancelDispute()` khi vụ việc còn hoạt động. Admin có thể `markUnderReview()` và `resolveDispute()` với kết quả `SELLER_WINS` hoặc `BUYER_WINS`. Seller thắng dẫn tới hoàn tất order và payout; buyer thắng dẫn tới hoàn toàn bộ giá chốt, hủy fulfillment và đưa sản phẩm về trạng thái trả lại. Enum có giá trị `REJECTED`, nhưng mã nguồn hiện không có chuyển tiếp nghiệp vụ đưa dispute tới trạng thái đó.

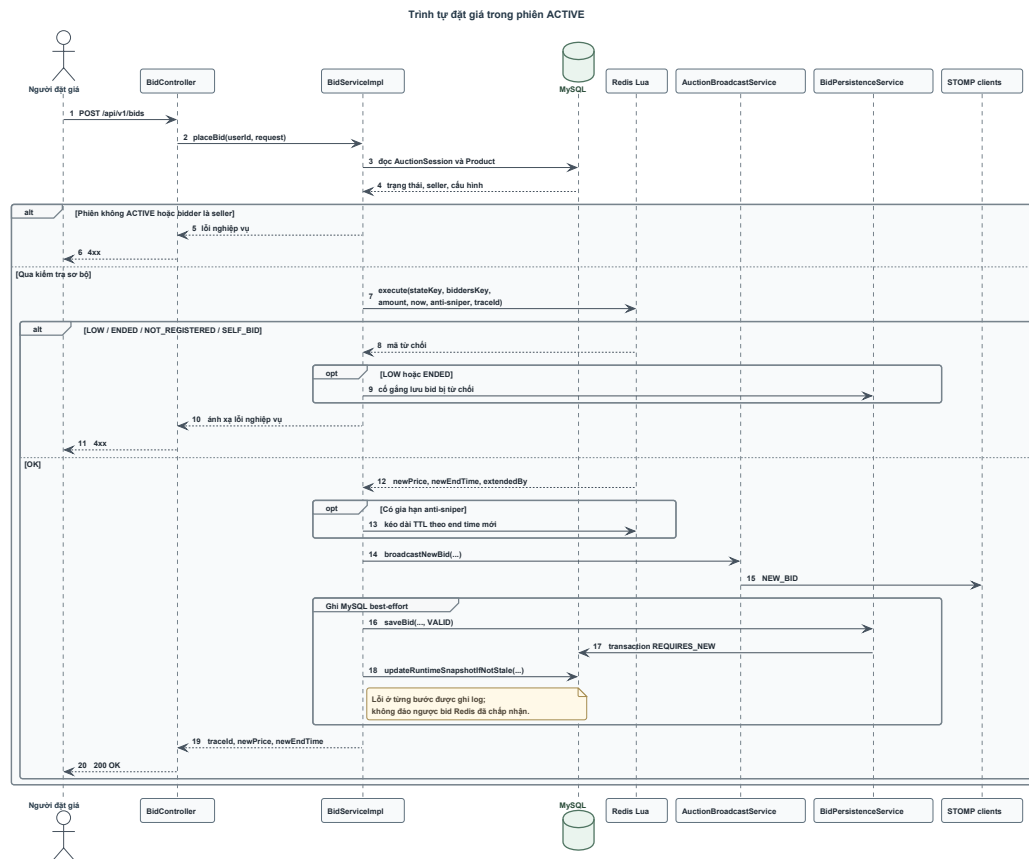


Hình 0.4: Bốn lớp điều phối nghiệp vụ tiêu biểu
 Nguồn: Tổng hợp từ các lớp dịch vụ backend.

e, Luồng trình tự đặt giá thời gian thực

Luồng bắt đầu khi trình duyệt gửi yêu cầu POST tới endpoint `/api/v1/bids`. Spring Security yêu cầu authority `CREATE_BID`; controller lấy user ID từ JWT và chuyển request tới `BidServiceImpl`. Service đọc `AuctionSession` cùng `Product` từ MySQL để kiểm tra trạng thái và quyền sở hữu, sau đó gọi Lua script với hash state và set bidder.

Lua script kiểm tra thời gian, tư cách participant, người đang dẫn đầu và giá tối thiểu. Nếu bid hợp lệ trong 30 giây cuối, script cộng 60 giây vào end time. Khi trả OK, backend phát `NEW_BID` trước, sau đó gọi tuần tự hai bước best-effort: lưu Bid bằng transaction `REQUIRES_NEW` và cập nhật snapshot session bằng câu lệnh chỉ chấp nhận dữ liệu không cũ hơn. Luồng cùng các nhánh từ chối được trình bày tại Hình ??.



Hình 0.5: Trình tự đặt giá trong phiên đang hoạt động

Nguồn: Tổng hợp từ *BidController*, *BidServiceImpl* and *Redis Lua*.

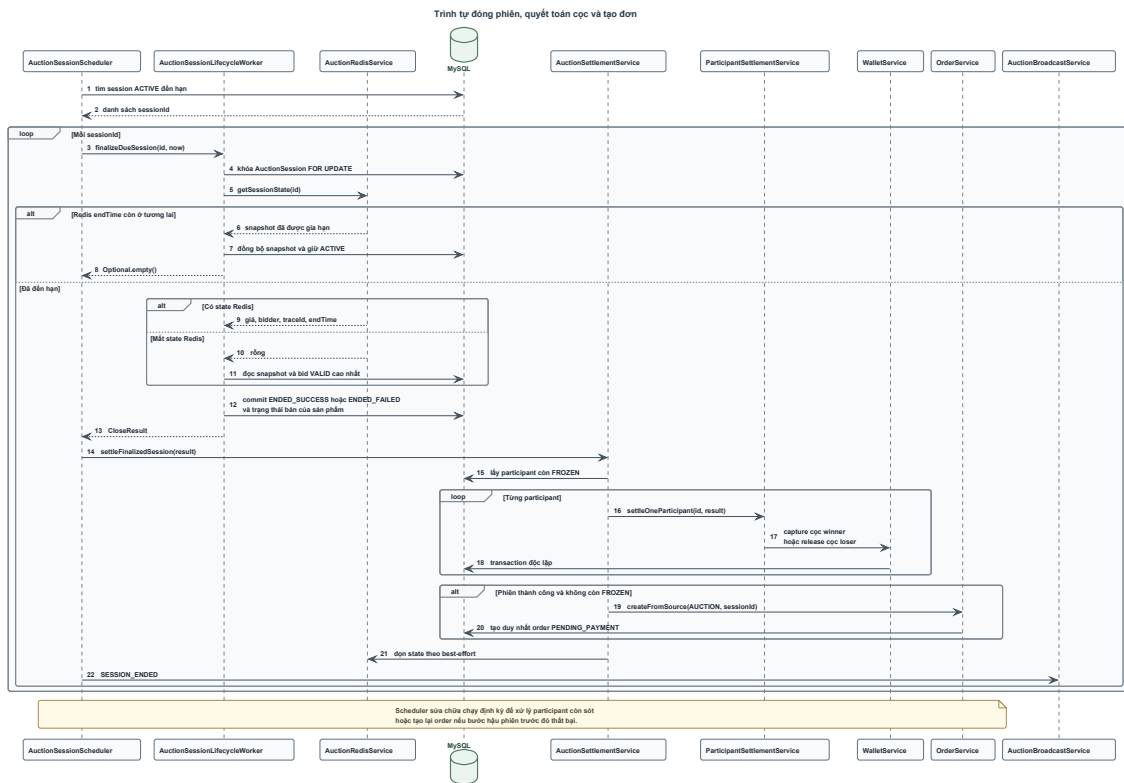
f, Luồng trình tự đóng phiên và tạo đơn

AuctionSessionScheduler chạy theo chu kỳ năm giây và lấy ID các phiên ACTIVE đến hạn theo snapshot MySQL. Với từng ID, scheduler gọi *finalizeDueSession()*. Worker khóa phiên và đối chiếu state Redis. Nếu end time Redis còn ở tương lai, phiên chưa bị đóng; nếu đã đến hạn, worker commit trạng thái terminal trước khi thực hiện thay đổi ví.

Sau khi nhận *CloseResult*, scheduler gọi settlement service. Service lấy participant còn FROZEN và gọi thành phần quyết toán riêng cho từng bản ghi. Mỗi participant được xử lý trong transaction *REQUIRES_NEW*: người thắng chuyển cọc sang DEDUCTED, người thua hoặc participant của phiên thất bại chuyển sang REFUNDED. Nếu một participant lỗi, participant khác vẫn được xử lý và scheduler sửa chữa thử lại về sau.

Khi phiên thành công và không còn participant FROZEN, service gọi *OrderService.createFromSource(AUCTION, sessionId)*. Order được tạo duy nhất ở trạng thái *PENDING_PAYMENT* với deadline 72 giờ. State Redis được dọn theo best-effort và scheduler phát *SESSION_ENDED*. Hình ?? thể hiện trình

tự này và cơ chế sửa chữa.



Hình 0.6: Trình tự đóng phiên, quyết toán cọc và tạo đơn
 Nguồn: Tổng hợp từ scheduler, lifecycle worker và settlement service.

g, Luồng thanh toán, giao nhận và hoàn tất

Người thắng chọn địa chỉ và thanh toán phần còn lại qua phương thức `payRemainder()`. Service xác minh quyền sở hữu order, deadline và số tiền; ví bị trừ phần còn lại; order chuyển sang `PAID`; fulfillment chờ giao được tạo với deadline 72 giờ. Nếu buyer không thanh toán đúng hạn, scheduler hủy order, chuyển cọc sang `CONFISCATED`, phân bổ 90% cọc cho seller và ghi 10% là doanh thu nền tảng.

Seller xác nhận giao hàng trước shipment deadline. Với giao hàng qua đơn vị thứ ba, carrier và tracking code là bắt buộc; với tự giao, hai trường này không bắt buộc. Order chuyển sang `FULLFILLING`, fulfillment chuyển `SHIPPED` và có auto-complete deadline sau 168 giờ. Buyer có thể xác nhận đã nhận để hoàn tất ngay; nếu không có tranh chấp, scheduler tự hoàn tất khi hết hạn. Khi hoàn tất, seller nhận giá chốt trừ commission và nền tảng ghi nhận commission.

Trong giai đoạn order `FULLFILLING` và fulfillment `SHIPPED`, buyer có thể mở tranh chấp. Vụ việc làm order chuyển `DISPUTED` và tạm dừng auto-complete. Admin giải quyết seller thắng để tiếp tục payout hoặc buyer thắng để hoàn lại toàn

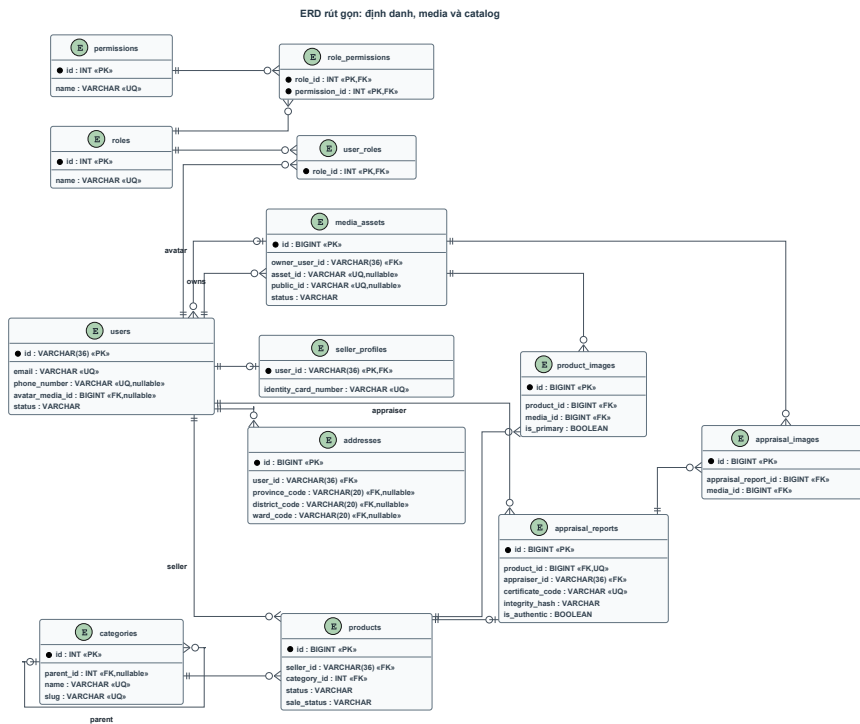
Nhóm dữ liệu	Các bảng chính	Số bảng
Identity, địa chỉ và quản trị	permissions, roles, role_permissions, users, user_roles, provinces, districts, wards, addresses, seller_profiles, các bảng token, capability và audit	15
Media	media_assets	1
Catalog và appraisal	categories, products, product_images, appraisal_reports, appraisal_images	5
Finance	wallets, wallet_transactions, wallet_operations, vnpay_deposits, platform_revenue_transactions	5
Auction	auction_sessions, auction_participants, bids	3
Order và fulfillment	orders, order_fulfillments	2
Dispute	dispute_cases, dispute_messages, dispute_evidence	3
Tổng cộng		34

Bảng 4: Phân nhóm các bảng dữ liệu của hệ thống

Lược đồ dữ liệu được trình bày theo ba cụm nghiệp vụ. Hình ?? mô tả nhóm định danh, media và catalog; Hình ?? mô tả nhóm tài chính và đấu giá; Hình ?? mô tả nhóm đơn hàng, giao nhận và tranh chấp.

Trong nhóm identity, `users` sử dụng UUID dạng chuỗi làm khóa chính và liên kết nhiều–nhiều với `roles` qua bảng liên kết. Quyền được ánh xạ từ role; trạng thái định chỉ `capability` được lưu riêng theo user. `seller_profiles` dùng `user_id` làm khóa chính, vì vậy mỗi user có tối đa một hồ sơ seller. Wallet được tạo theo nhu cầu nên một user có từ không đến một wallet; khóa ngoại user trong bảng wallet có ràng buộc unique.

Trong nhóm catalog, một seller có thể sở hữu nhiều product. Một product có nhiều `product_images`, nhưng service yêu cầu đúng một ảnh chính khi tạo hoặc cập nhật. `appraisal_reports.product_id` có ràng buộc unique, tạo quan hệ product 1–0..1 report. `certificate_code` và `integrity_hash` phục vụ tra cứu cùng kiểm tra vân tay dữ liệu; chúng không phải chữ ký số pháp lý.

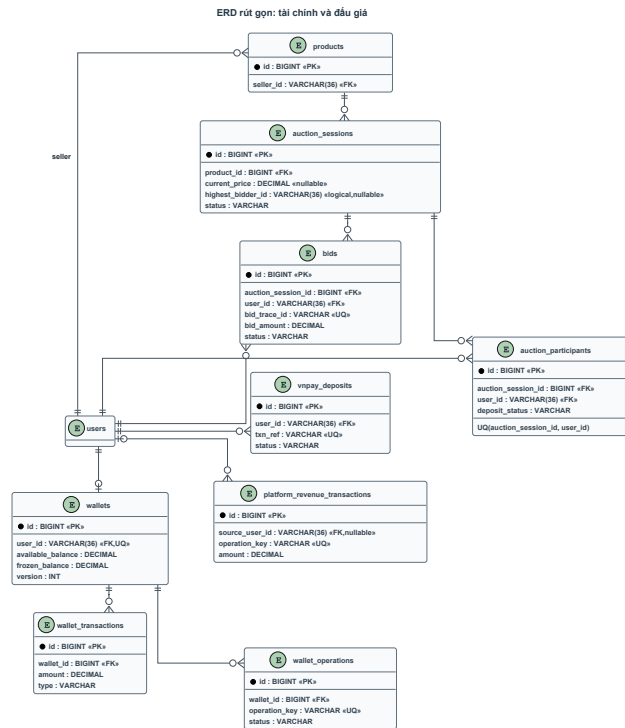


Hình 0.8: ERD rút gọn của nhóm định danh, media và catalog
 Nguồn: Tổng hợp từ Flyway migration V1-V5.

Biểu đồ ERD này làm rõ sự cô lập dữ liệu giữa tài khoản người dùng thông thường và hồ sơ người bán, đồng thời chuẩn hóa thông tin địa chỉ giao nhận qua các mã địa giới hành chính. Hệ thống đảm bảo tính toàn vẹn tham chiếu thông qua các khóa ngoại vật lý và composite primary key ở các bảng liên kết phân quyền nhiều-nhiều. Ngoài ra, cấu trúc của bảng `seller_profiles` và `addresses` được thiết kế linh hoạt, cho phép lưu trữ chi tiết các thông tin pháp lý phục vụ cho việc hậu kiểm và đối soát của quản trị viên. Việc tách biệt bảng chứa tập tin phương tiện `media_assets` giúp quản lý tập trung và tối ưu hóa việc phân phát tài nguyên hình ảnh qua dịch vụ đám mây Cloudinary.

Trong nhóm finance, `wallet_transactions` lưu các biến động đã thành công; `wallet_operations` quản lý vòng đời thao tác và bảo đảm `operation_key` là duy nhất. `platform_revenue_transactions` ghi phí thẩm định, hoa hồng và phần phí từ cọc bị tịch thu. Không có sổ cái ký quỹ riêng.

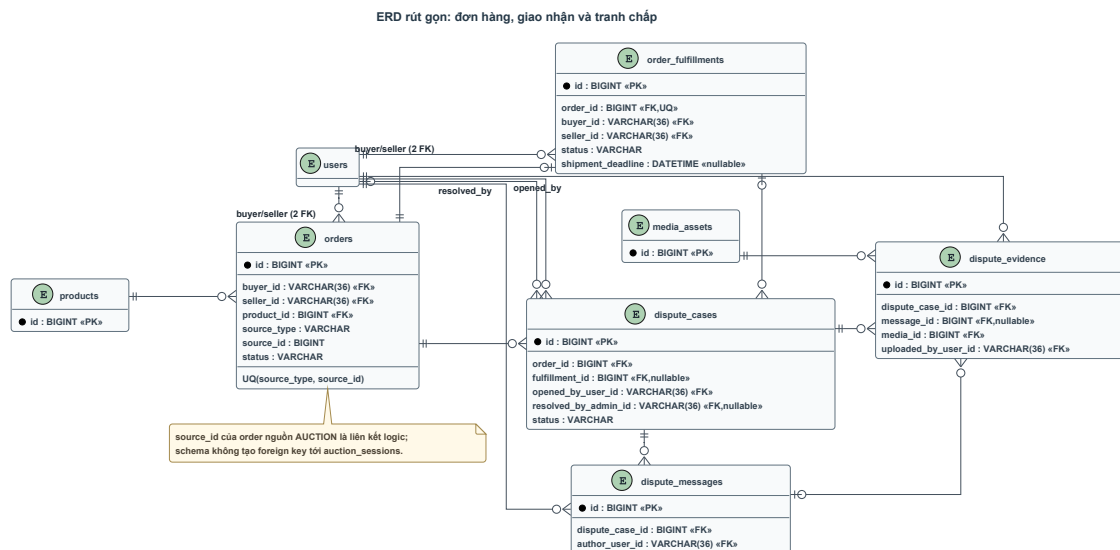
Một product có thể có nhiều `auction_sessions` trong lịch sử, nhưng service chỉ cho phép một phiên WAITING hoặc ACTIVE tại một thời điểm. Quy tắc này được thực thi bằng khóa và kiểm tra nghiệp vụ, không phải unique constraint theo trạng thái. `auction_participants` có unique constraint trên cặp `(auction_session_id, user_id)`; `bids.bid_trace_id` là duy nhất và bid được xem là audit append-only.



Hình 0.9: ERD rút gọn của nhóm tài chính và đấu giá
Nguồn: Tổng hợp từ Flyway migration V1 – V5.

Sơ đồ tài chính thể hiện rõ tính độc lập của các thao tác ví thông qua khóa duy nhất của bảng operation, giúp ngăn ngừa việc thực thi lặp và bảo vệ số dư khả dụng của tài khoản. Quan hệ giữa các bảng đấu giá và lịch sử bid được tối ưu hóa cho việc truy vết và kiểm toán các giao dịch ký quỹ. Cơ chế ghi nhật ký giao dịch trong bảng wallet_transactions hoạt động theo nguyên tắc append-only, đảm bảo mọi biến động số dư đều được lưu vết vĩnh viễn và không thể bị sửa đổi trái phép. Ràng buộc duy nhất trên cặp khóa ngoại của bảng đăng ký tham gia cũng giúp ngăn chặn việc một người dùng đăng ký nhiều lần trong cùng một phiên đấu giá, giữ vững tính công bằng của hệ thống.

orders dùng cặp (source_type, source_id) làm khóa duy nhất. Với nguồn AUCTION, source ID trở logic tới session nhưng schema không tạo foreign key trực tiếp từ order sang auction_sessions; adapter chịu trách nhiệm xác minh nguồn. Mỗi order có tối đa một order_fulfillments. Một order có thể có nhiều dispute theo lịch sử, nhưng service chỉ ngăn nhiều dispute hoạt động đồng thời. dispute_evidence luôn thuộc một dispute và có thể không gắn message nếu là bằng chứng lúc mở vụ việc.



Hình 0.10: ERD rút gọn của nhóm đơn hàng, giao nhận và tranh chấp

Nguồn: Tổng hợp từ Flyway migration V1–V5.

Thiết kế cơ sở dữ liệu của phân hệ sau đấu giá tập trung vào tính nhất quán trạng thái giữa đơn hàng, thông tin giao nhận và tranh chấp phát sinh. Các mối liên kết này hỗ trợ đặc lực cho việc xử lý khiếu nại và tự động hoàn tất đơn hàng theo thời gian định trước. Sự vắng mặt của khóa ngoại vật lý giữa bảng `orders` và `auction_sessions` được bù đắp bằng các ràng buộc logic chặt chẽ trong mã nguồn ứng dụng, giúp giảm thiểu sự phụ thuộc trực tiếp giữa phân hệ đấu giá và phân hệ sau đấu giá, tạo điều kiện thuận lợi cho việc bảo trì và mở rộng hệ thống sau này. Các bằng chứng tranh chấp trong bảng `dispute_evidence` được liên kết chặt chẽ với vụ việc tranh chấp để làm căn cứ pháp lý rõ ràng cho admin khi ra quyết định phân xử.

Redis không được đưa vào ERD vì không phải nguồn dữ liệu quan hệ bền vững. Hai key chính của phiên hoạt động là `auction:session:{id}:state` và `auction:session:{id}:bidders`. Production bật AOF và volume nhằm giảm mất state runtime khi container khởi động lại, nhưng Redis vẫn là nguồn trạng thái runtime của phiên ACTIVE, còn MySQL quản lý dữ liệu bền vững và trạng thái terminal.

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Chương ?? đã trình bày cơ sở lựa chọn công nghệ. Phần này liệt kê các phiên bản đang được Maven và `pnpm-lock.yaml` sử dụng. Công cụ phía backend được trình bày tại Bảng ??; công cụ phía frontend, kiểm thử và triển khai được trình bày tại Bảng ??.

Cloudinary, VNPay Sandbox và SMTP được tích hợp qua HTTP hoặc giao thức thư điện tử. Frontend tải trực tiếp tệp lên Cloudinary bằng upload intent có chữ ký; dự án không phụ thuộc Cloudinary SDK trong manifest hiện tại. VNPay được dùng cho luồng nạp tiền thử nghiệm; SMTP phục vụ xác thực email và khôi phục mật khẩu.

Mục đích	Công cụ hoặc thư viện	Phiên bản	Địa chỉ chính thức
Nền tảng backend	Java Development Kit	17	oracle.com/java
Khung ứng dụng	Spring Boot	3.5.15	spring.io
Truy cập dữ liệu	Spring Data JPA / Hibernate	3.5.12 / 6.6.53.Final	spring.io
Truy cập Redis	Spring Data Redis	3.5.12	spring.io
Bảo mật	Spring Security	6.5.11	spring.io
Di trú schema	Flyway	11.7.2	red-gate.com
Ảnh xạ DTO	MapStruct	1.5.5.Final	mapstruct.org
Giảm mã lập Java	Lombok	1.18.46	projectlombok.org
Unit test backend	JUnit Jupiter / Mockito	5.12.2 / 5.17.0	junit.org
Integration test	Testcontainers	1.21.4	testcontainers.com
Cơ sở dữ liệu	MySQL	8.0	dev.mysql.com
Trạng thái runtime	Redis	7.4-alpine	redis.io

Bảng 5: Thư viện và công cụ backend

Mục đích	Công cụ hoặc thư viện	Phiên bản	Địa chỉ chính thức
Xây dựng SPA	React / React DOM	19.2.6	react.dev
Ngôn ngữ frontend	TypeScript	5.9.3	typescriptlang.org
Build frontend	Vite	7.3.3	vite.dev
Điều hướng SPA	React Router	7.15.0	reactrouter.com
Server state	TanStack React Query	5.100.9	tanstack.com
Auth state phía client	Zustand	5.0.13	zustand.pmnd.rs
HTTP client	Axios	1.16.0	axios-http.com
CSS và design token	Tailwind CSS	4.2.4	tailwindcss.com
Form và validation	React Hook Form / Zod	7.75.0 / 4.4.3	react-hook-form.com
Giao tiếp STOMP	STOMP.js / SockJS Client	7.3.0 / 1.6.1	stomp-js.github.io
Unit test frontend	Vitest	4.1.5	vitest.dev
E2E trình duyệt	Playwright	1.59.1	playwright.dev
Đóng gói và điều phối	Docker / Docker Compose	25.0.3	docs.docker.com
Proxy và phân phát SPA	Nginx	1.24.0	nginx.org

Bảng 6: Thư viện và công cụ frontend, kiểm thử và triển khai

0.3.2 Kết quả đạt được

Kết quả xây dựng gồm ứng dụng backend, ứng dụng frontend và bộ cấu hình triển khai production. Backend cung cấp API cho xác thực, hồ sơ và địa chỉ, seller, media, sản phẩm, thẩm định, chứng thư, ví, VNPay, đấu giá, đặt giá, đơn hàng, giao nhận, tranh chấp và quản trị. Frontend cung cấp khu public, buyer, seller, appraiser, admin cùng phòng đặt giá toàn màn hình. Các luồng chính từ đăng ký tài khoản, đưa sản phẩm đi thẩm định, tạo phiên, ký quỹ, đặt giá, thanh toán, giao hàng tới giải quyết tranh chấp đều có API và giao diện tương ứng.

Ứng dụng được đóng gói thành hai image do dự án xây dựng. Image backend dùng quy trình nhiều giai đoạn: Maven tạo JAR trên image build, sau đó JAR chạy

bằng JRE 17 với user không đặc quyền. Image frontend dùng Node và pnpm để build SPA, rồi sao chép thư mục `dist` sang Nginx. Docker Compose ghép hai image này với MySQL và Redis, đồng thời cấu hình health check và volume.

Bảng ?? thống kê số tệp chính và số bảng dữ liệu của hệ thống tại phiên bản hiện tại. Số dòng mã không được sử dụng vì chưa có kết quả đo bằng công cụ chuyên dụng.

Chỉ số	Giá trị	Căn cứ
File Java production	355	<code>src/main/java</code>
Entity Java	59	Các package entity
Controller backend	25	Các package controller
File test Java	64	<code>src/test/java</code>
File TypeScript/TSX production	262	<code>woodcert-auction-fe/src</code> , loại file test
Trang giao diện theo quy ước <code>*Page.tsx</code>	47	<code>src/features</code>
File unit test frontend	59	Các file <code>*.test.ts</code> và <code>*.test.tsx</code>
Bảng MySQL	34	Flyway V1 và V4
Migration Flyway	5	V1-V5

Bảng 7: Thống kê quy mô mã nguồn và cơ sở dữ liệu

Kết quả build và test được trình bày tại Mục ?. Hệ thống đã được đưa lên production và hoạt động ổn định trong quá trình sử dụng. Trang chủ HTTPS cùng endpoint readiness đều trả HTTP 200 tại lần kiểm tra gần nhất.

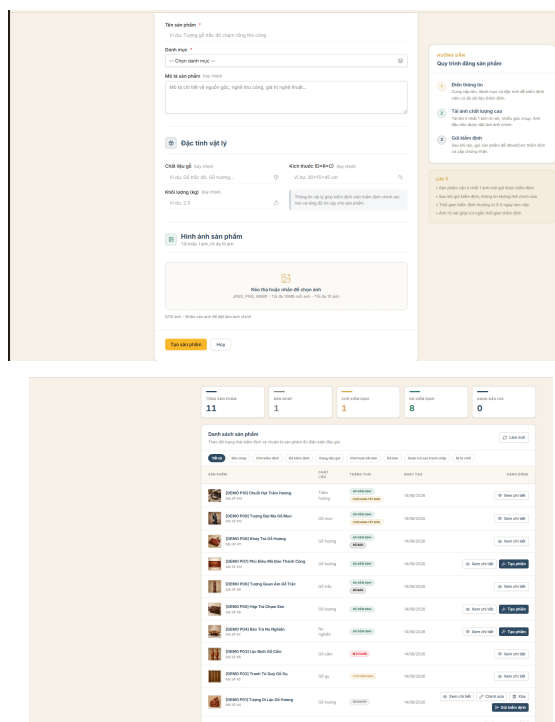
0.3.3 Minh họa các chức năng chính

Các ảnh minh họa ở phần này được chụp trực tiếp từ ứng dụng. Phần hiển thị tài khoản và số dư không cần thiết đã được cắt bỏ hoặc che; ảnh không chứa token, khóa API hay thông tin đăng nhập.

a, Tạo sản phẩm và gửi thẩm định

Khu vực dành cho người bán được thiết kế tối giản để hỗ trợ quá trình đăng tải sản phẩm gỗ mỹ nghệ một cách nhanh chóng. Người bán có thể tạo bản nháp sản phẩm, nhập các thông tin mô tả chi tiết về kích thước, chất liệu gỗ, độ tuổi của gỗ, nguồn gốc xuất xứ và các tài liệu chứng minh tính hợp pháp đi kèm. Hệ thống hỗ trợ tải lên nhiều hình ảnh chụp chi tiết các góc cạnh của sản phẩm qua dịch vụ đám mây Cloudinary. Người bán bắt buộc phải lựa chọn đúng một ảnh đại diện chính làm ảnh hiển thị chính trong danh mục đầu giá trước khi có thể gửi yêu cầu thẩm định.

Trong giai đoạn sản phẩm còn ở trạng thái DRAFT, người bán hoàn toàn có quyền chỉnh sửa hoặc xóa thông tin sản phẩm. Khi người bán bấm nút gửi yêu cầu thẩm định, hệ thống sẽ thực hiện xác minh số dư tài khoản để tự động khấu trừ một khoản phí thẩm định cố định từ ví cá nhân của người bán. Khi giao dịch tài chính thành công, trạng thái sản phẩm sẽ tự động chuyển sang chờ thẩm định (UNDER_APPRAISAL) và khóa toàn bộ quyền chỉnh sửa của người bán để bảo đảm tính toàn vẹn dữ liệu trước khi thẩm định viên tiếp nhận. Quy trình này được minh họa trực quan tại Hình ??.



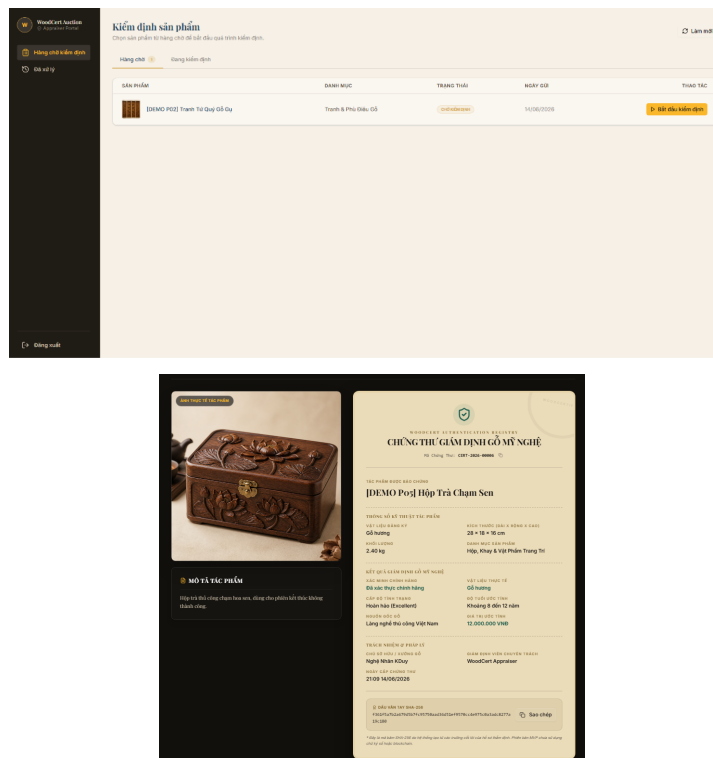
Hình 0.11: Biểu mẫu tạo sản phẩm và danh sách theo dõi trạng thái thẩm định

Nguồn: Ảnh chụp giao diện WoodCert Auction.

b, Kiểm định và tra cứu chứng thư

Thẩm định viên sử dụng một cổng thông tin chuyên biệt để theo dõi danh sách các sản phẩm đang xếp hàng chờ kiểm định từ toàn bộ người bán trên hệ thống. Khi tiếp nhận một hồ sơ, sản phẩm sẽ được khóa cho riêng thẩm định viên đó xử lý. Giao diện làm việc của thẩm định viên cho phép nhập các đánh giá chuyên môn chi tiết về loại gỗ, tình trạng nguyên vẹn, tuổi thọ ước tính của gỗ, và kết luận xác thực sản phẩm. Thẩm định viên bắt buộc phải tải lên các hình ảnh bằng chứng kiểm định cận cảnh các chi tiết vân gỗ hoặc các vết chạm trổ đặc trưng để làm căn cứ pháp lý. Sau khi báo cáo được gửi, hệ thống tự động khóa và không cho phép chỉnh sửa hay xóa để tránh gian lận, đồng thời chuyển trạng thái sản phẩm sang đã thẩm định (APPRAISED) hoặc từ chối (REJECTED).

Bên cạnh đó, bất kỳ người dùng nào cũng có thể tra cứu thông tin chứng thư công khai thông qua mã số chứng thư độc nhất được cấp cho sản phẩm. Trang tra cứu chứng thư hiển thị toàn bộ thông tin chi tiết của sản phẩm gỗ, báo cáo kết luận của thẩm định viên và mã vân tay SHA-256 tham chiếu của dữ liệu báo cáo thẩm định. Việc cung cấp mã hash SHA-256 giúp người mua hoặc bên thứ ba có thể dễ dàng đối chiếu, xác minh tính toàn vẹn và chống làm giả chứng thư ngoại tuyến. Giao diện hàng chờ thẩm định và tra cứu chứng thư được mô tả tại Hình ??.



Hình 0.12: Hàng chờ thẩm định và trang tra cứu chứng thư công khai

Nguồn: Ảnh chụp giao diện WoodCert Auction.

c, Tạo phiên và đăng ký tham gia

Chức năng tạo phiên đấu giá chỉ khả dụng đối với những sản phẩm đã có báo cáo thẩm định hợp lệ ở trạng thái APPRAISED và chưa từng tham gia phiên đấu giá nào khác (AVAILABLE). Biểu mẫu tạo phiên đấu giá yêu cầu người bán thiết lập các tham số vận hành quan trọng bao gồm: thời gian bắt đầu và kết thúc phiên, mức giá khởi điểm, giá bảo lưu (reserve price - mức giá tối thiểu mà người bán chấp nhận bán sản phẩm), bước giá tối thiểu cho mỗi lượt bid, và số tiền đặt cọc yêu cầu đối với người tham gia. Giao diện biểu mẫu tích hợp sẵn các công cụ gợi ý thông minh dựa trên giá trị thẩm định của sản phẩm để đề xuất mức giá khởi điểm và tiền cọc tối thiểu phù hợp, giúp tăng xác suất đấu giá thành công và bảo đảm quyền lợi tài chính cho người bán.

Về phía người mua, để tham gia đấu giá trong một phiên cụ thể, người mua bắt buộc phải đăng ký tham gia trước khi phiên kết thúc. Quá trình đăng ký yêu cầu hệ thống thực hiện kiểm tra số dư ví khả dụng của người mua; nếu số dư ví lớn hơn hoặc bằng số tiền cọc yêu cầu của phiên đấu giá, hệ thống sẽ thực hiện đóng băng khoản tiền này trong ví của người mua và chuyển trạng thái đăng ký sang thành công. Số tiền đóng băng này hoạt động như một khoản ký quỹ để ràng buộc trách nhiệm tham gia đấu giá của người mua. Giao diện tạo phiên đấu giá được thể hiện chi tiết tại Hình ??.

The screenshot shows a web form titled 'Sản phẩm đưa lên đấu giá' (Product to auction). It includes a dropdown for 'Sản phẩm đã chọn' (Selected product) with a value of 'IDMAG F01 Phụ kiện H&B Sơn Thanh Công'. Below this is a 'Giá và tiền cọc' (Price and deposit) section with a table showing 'Giá khởi điểm' (Starting price) at 100,000, 'Giá sàn' (Floor price) at 1,000,000, and 'Tiền cọc' (Deposit) at 1,000,000. The 'Thời gian phiên' (Auction time) section shows 'Bắt đầu' (Start) at 23/10/2020 09:34:54 and 'Kết thúc' (End) at 23/10/2020 10:34:54. There are buttons for 'Tạo phiên đấu giá' (Create auction) and 'Hủy' (Cancel).

Hình 0.13: Biểu mẫu tạo phiên đấu giá từ sản phẩm đã được thẩm định

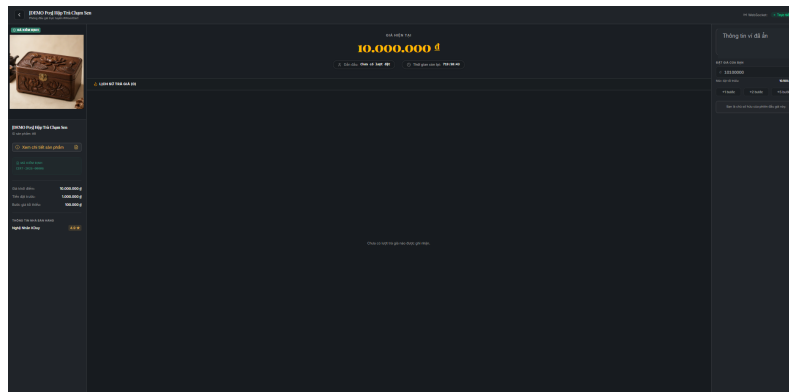
Nguồn: Ảnh chụp giao diện WoodCert Auction.

d, Phòng đặt giá thời gian thực

Phòng đấu giá trực tuyến được thiết kế theo giao diện bảng điều khiển cockpit toàn màn hình nhằm tối ưu hóa sự tập trung của người dùng vào các biến động giá thời gian thực. Giao diện hiển thị trực quan mức giá hiện tại, định danh người đang dẫn đầu (đã được ẩn danh bằng cách che một phần ký tự để bảo đảm tính riêng tư), đồng hồ đếm ngược đồng bộ chính xác với thời gian của máy chủ backend, lịch sử chi tiết của tất cả các lượt đặt giá thành công và trạng thái kết nối thời gian thực qua giao thức WebSocket. Người dùng chỉ có thể thực hiện thao tác đặt giá khi đã đăng ký tham gia phiên thành công và trạng thái phiên hiển thị hợp lệ với thuộc tính `canBid=true` do máy chủ trả về.

Khi có bất kỳ lượt đặt giá mới nào được máy chủ chấp nhận, thông tin phòng đấu giá sẽ lập tức được cập nhật tức thời tới toàn bộ các trình duyệt đang kết nối mà không cần tải lại trang. Hệ thống tích hợp sẵn các thông báo đẩy (toast notifications) để cảnh báo tức thì cho người dùng khi họ bị người khác vượt giá, giúp họ nhanh chóng đưa ra quyết định đặt giá tiếp theo. Ngoài ra, cơ chế chống đặt giá giây cuối (anti-sniper) hoạt động đồng bộ với backend; nếu có lượt đặt giá hợp lệ trong 30 giây cuối cùng, thời điểm đóng phiên sẽ tự động kéo dài thêm 60

giây, và giao diện sẽ hiển thị một banner thông báo màu vàng nổi bật để cảnh báo tất cả người tham gia về sự gia hạn này. Bố cục của phòng đấu giá trực tuyến được minh họa tại Hình ??.



Hình 0.14: Phòng đặt giá khi phiên đấu giá đang hoạt động

Nguồn: Ảnh chụp giao diện WoodCert Auction.

e, Thanh toán và giao nhận

Khi phiên đấu giá kết thúc thành công, hệ thống tự động khởi tạo một đơn hàng mới ở trạng thái chờ thanh toán (PENDING_PAYMENT) dành cho người thắng cuộc. Người mua có thời hạn 72 giờ để truy cập vào chi tiết đơn hàng, lựa chọn địa chỉ giao nhận và thực hiện thanh toán phần giá trị còn lại (giá chốt trừ đi khoản tiền đặt cọc đã ký quỹ trước đó). Sau khi người mua hoàn tất thanh toán, trạng thái đơn hàng chuyển sang đã thanh toán (PAID), đồng thời gửi thông báo yêu cầu người bán chuẩn bị và giao hàng cho đơn vị vận chuyển.

Người bán có trách nhiệm xác nhận giao hàng cho đơn vị vận chuyển trước thời hạn giao hàng (shipment deadline). Khi xác nhận giao hàng, đối với các đơn vị vận chuyển bên thứ ba, người bán bắt buộc phải nhập tên nhà vận chuyển và mã số vận đơn thực tế để làm cơ sở đối soát. Trạng thái đơn hàng khi đó sẽ chuyển sang đang giao (FULFILLING) và người mua có thể theo dõi trực tiếp mã vận đơn để chủ động liên hệ nhận hàng. Hệ thống cũng thiết lập thời hạn tự động hoàn tất đơn hàng sau 168 giờ kể từ khi người bán giao hàng, nếu người mua không xác nhận đã nhận và không có tranh chấp phát sinh. Giao diện danh sách đơn bán của người bán được minh họa tại Hình ??.

Trạng thái	Đơn hàng	Giá chào	Vận chuyển	Chức năng
Đã thanh toán	Đơn #6: CHAI SƠN THANH TRẦN (phần còn lại) IDEMO P001 Chậu Hết Trầm Hương Phân 01 - Ngày bán: 22/07/2025 14:00:00	10.003.000.000 VND	Chưa tạo	Chi tiết
Đã hủy	Đơn #5: ĐEMO P001 Chậu Hết Trầm Hương Phân 01 - Ngày bán: 22/07/2025 14:00:00	3.200.000 VND	Chưa tạo	Chi tiết
Đang chờ thanh toán	Đơn #4: ĐEMO P001 Chậu Hết Trầm Hương Phân 01 - Ngày bán: 22/07/2025 14:00:00	3.200.000 VND	Đang vận chuyển	Chi tiết
Đã thanh toán	Đơn #3: ĐEMO P001 Chậu Hết Trầm Hương Phân 01 - Ngày bán: 22/07/2025 14:00:00	3.200.000 VND	Tự động hoàn tất	Chi tiết
Đã thanh toán	Đơn #2: ĐEMO P001 Chậu Hết Trầm Hương Phân 01 - Ngày bán: 22/07/2025 14:00:00	3.200.000 VND	Đã hủy vận chuyển	Chi tiết
Đã thanh toán	Đơn #1: ĐEMO P001 Chậu Hết Trầm Hương Phân 01 - Ngày bán: 22/07/2025 14:00:00	3.200.000 VND	Đã hủy vận chuyển	Chi tiết

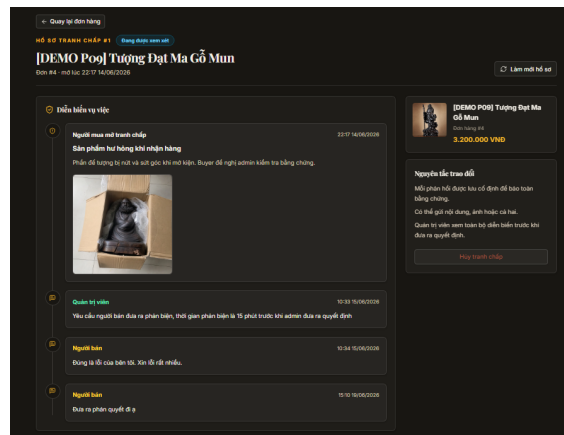
Hình 0.15: Danh sách đơn bán theo trạng thái thanh toán và giao nhận

Nguồn: Ảnh chụp giao diện WoodCert Auction.

f, Tranh chấp và quản trị

Trong trường hợp sản phẩm nhận được bị hư hại do vận chuyển hoặc không đúng với mô tả trong báo cáo thẩm định ban đầu, người mua có quyền gửi yêu cầu khiếu nại hoặc mở tranh chấp đơn hàng. Chức năng mở tranh chấp chỉ khả dụng khi đơn hàng đang ở trạng thái giao hàng và thông tin vận chuyển thực tế đã ở trạng thái SHIPPED. Người mua bắt buộc phải mô tả chi tiết lý do tranh chấp và tải lên các bằng chứng ban đầu bao gồm hình ảnh hoặc video mở hộp sản phẩm để làm cơ sở pháp lý cho yêu cầu của mình. Khi tranh chấp được mở, hệ thống sẽ tạm dừng toàn bộ tiến trình tự động hoàn tất đơn hàng và đóng băng khoản thanh toán để bảo đảm an toàn tài chính.

Giao diện hồ sơ tranh chấp hoạt động như một kênh trao đổi thông tin trực tiếp ba bên giữa người mua, người bán và quản trị viên hệ thống. Các bên có thể gửi phản hồi, tranh luận và tiếp tục bổ sung thêm các hình ảnh bằng chứng trực quan trong cùng một luồng hội thoại. Quản trị viên hệ thống sẽ đóng vai trò là trọng tài xem xét toàn bộ diễn biến trao đổi và các bằng chứng thực tế được cung cấp để ra phán quyết cuối cùng: hoặc chấp nhận tranh chấp của người mua để hoàn trả tiền, hoặc bác bỏ tranh chấp để tiến hành quyết toán đơn hàng cho người bán. Giao diện chi tiết của hồ sơ tranh chấp và lịch sử trao đổi được thể hiện tại Hình ??.



Hình 0.16: Hồ sơ tranh chấp với bằng chứng và trao đổi giữa các bên

Nguồn: Ảnh chụp giao diện WoodCert Auction.

0.4 Kiểm thử

WoodCert Auction được kiểm tra ở các mức đơn vị, tích hợp, giao diện, đầu cuối và đóng gói. Bộ kiểm thử được chạy trên phiên bản 5aa5a4770db4 ngày 22/06/2026. Kết quả phản ánh những kịch bản đã thực thi, không thay thế cho kiểm thử tải hoặc quá trình theo dõi production dài hạn.

Môi trường backend sử dụng Windows 11, Java 17.0.12, Maven Wrapper 3.9.14 và Docker Server 25.0.3. Testcontainers khởi tạo MySQL và Redis thật trong container cho bài integration. Frontend sử dụng Node.js 20.19.0, pnpm 11.0.9, Vitest 4.1.5 và Playwright 1.59.1 với Chromium.

0.4.1 Phương pháp và kịch bản kiểm thử

Kiểm thử đơn vị backend cô lập service, policy, mapper và validator bằng JUnit/Mockito. Nhóm này tập trung vào biên giá trị, chuyển trạng thái, quyền sở hữu, operation key và hành vi khi dependency trả lỗi. Kiểm thử tích hợp sử dụng Testcontainers để xác minh Flyway V1-V5, ràng buộc schema, Redis Lua, bảo mật API và cấu hình runtime gần production.

Frontend dùng Vitest cùng Testing Library để kiểm tra API mapper, hook, route guard, form và component theo trạng thái. Playwright chạy trình duyệt thật cho một số kịch bản giao diện trên desktop và mobile. CI đồng thời chạy ESLint, TypeScript typecheck và production build.

Bảng ?? trình bày các ca đại diện của ba nhóm nghiệp vụ trọng tâm. Danh sách đầy đủ nằm trong mã kiểm thử và báo cáo Surefire, Vitest, Playwright.

Mã	Kịch bản đại diện	Kết quả mong đợi	Căn cứ
BID-01	Participant hợp lệ đặt giá đạt bước giá trong phiên ACTIVE	Redis cập nhật nguyên tử giá và người dẫn đầu; backend phát sự kiện và ghi MySQL best-effort	Lua integration, service test
BID-02	Giá thấp, tự đặt giá hoặc chưa đăng ký	Từ chối và không thay đổi trạng thái Redis	Lua integration
BID-03	Bid hợp lệ trong 30 giây cuối	End time tăng 60 giây và TTL được kéo dài	Lua và runtime flow
BID-04	Redis đã nhận bid nhưng bước MySQL phụ trợ lỗi	Không đảo ngược bid; API thành công và ghi log lỗi	Service unit test
FIN-01	Gọi lại cùng operation key với cùng nội dung	Không tạo thêm biến động sổ dư	Wallet tests
FIN-02	Đóng phiên và quyết toán participant	Winner DEDUCTED, loser REFUNDED; lỗi cục bộ không chặn participant khác	Settlement tests
ORD-01	Tạo order hoặc quá hạn thanh toán 72 giờ	Tạo duy nhất một order; khi quá hạn thì hủy và phân bổ cọc	Order tests
FUL-01	Seller giao hợp lệ hoặc quá hạn giao	Chuyển SHIPPED; nếu quá hạn thì hoàn tiền và trả sản phẩm về AVAILABLE	Fulfillment tests
FUL-02	Hết 168 giờ, không có tranh chấp	Tự hoàn tất order và payout seller	Scheduler test
DSP-01	Mở hoặc giải quyết tranh chấp	Yêu cầu bằng chứng; kết quả cập nhật nhất quán order, fulfillment và dòng tiền	Dispute tests

Bảng 8: Các ca kiểm thử nghiệp vụ đại diện

0.4.2 Kết quả kiểm thử tự động

Backend được chạy bằng Maven Wrapper ở chế độ test. Maven Surefire tạo 63 report XML với tổng cộng 403 test, không có failure, error hoặc skipped. Suite gồm kiểm thử migration Flyway, Redis Lua, bảo mật đầu giá và cấu hình runtime production, không chỉ gồm unit test dùng mock.

Frontend được kiểm tra bằng `pnpm lint`, `pnpm typecheck`, `pnpm test` và `pnpm build`. Vitest chạy 196 test trong 59 file, toàn bộ đạt. Production build biến đổi 2.174 module và tạo thành công thư mục `dist`. Playwright chạy riêng bằng `pnpm test:e2e`, gồm 7 test trong 5 spec và đều đạt.

Bảng ?? tổng hợp số lượng ca thực thi và kết quả của từng nhóm kiểm thử, kiểm tra mã nguồn và bước đóng gói.

Phạm vi	Lệnh hoặc báo cáo	Số lượng	Đạt	Lỗi
Backend unit và integration	Maven Surefire XML	403 test / 63 suite	403	0
Frontend unit/component	Vitest	196 test / 59 file	196	0
Frontend E2E	Playwright Chromium	7 test / 5 spec	7	0
Kiểm tra mã frontend	ESLint	Toàn bộ mã nguồn	Đạt	0
Kiểm tra kiểu	TypeScript	Toàn bộ mã nguồn	Đạt	0
Đóng gói frontend	Vite production build	2.174 module	Đạt	0
Biên dịch backend	Maven compile	Toàn bộ mã nguồn	Đạt	0

Bảng 9: Kết quả kiểm thử và kiểm tra build

Toàn bộ các ca trong bộ kiểm thử đều đạt ở môi trường nêu trên. Dự án chưa thực hiện benchmark tải nên chưa có số liệu về p95, thông lượng hoặc tỷ lệ sẵn sàng. Việc production vận hành ổn định trong quá trình sử dụng là kết quả thực tế, nhưng chưa thay thế được dữ liệu giám sát dài hạn hoặc biên bản nghiệm thu cho từng kịch bản.

0.4.3 Kiểm tra trên môi trường production

Ngày 22/06/2026, truy cập HTTP tới `woodauction.id.vn` được chuyển hướng 301 sang HTTPS; trang chủ và endpoint readiness đều trả mã 200. Chứng thư TLS của tên miền hợp lệ và Nginx phục vụ được cả chuyển hướng lẫn kết nối HTTPS.

Kiểm thử E2E tập trung vào trang gốc, đường dẫn từ chi tiết đấu giá tới phòng đặt giá, trang không tìm thấy của từng portal và trang tranh chấp trên desktop/mobile. Các chuỗi nạp tiền VNPay, nhiều người đặt giá đồng thời, gửi email thực, tải tệp lên Cloudinary và giao nhận ngoài thực tế chưa được tự động hóa trọn vẹn. Những chức năng này đã có trong hệ thống nhưng chưa có bộ kiểm thử nghiệm thu đầu cuối đầy đủ.

0.5 Triển khai

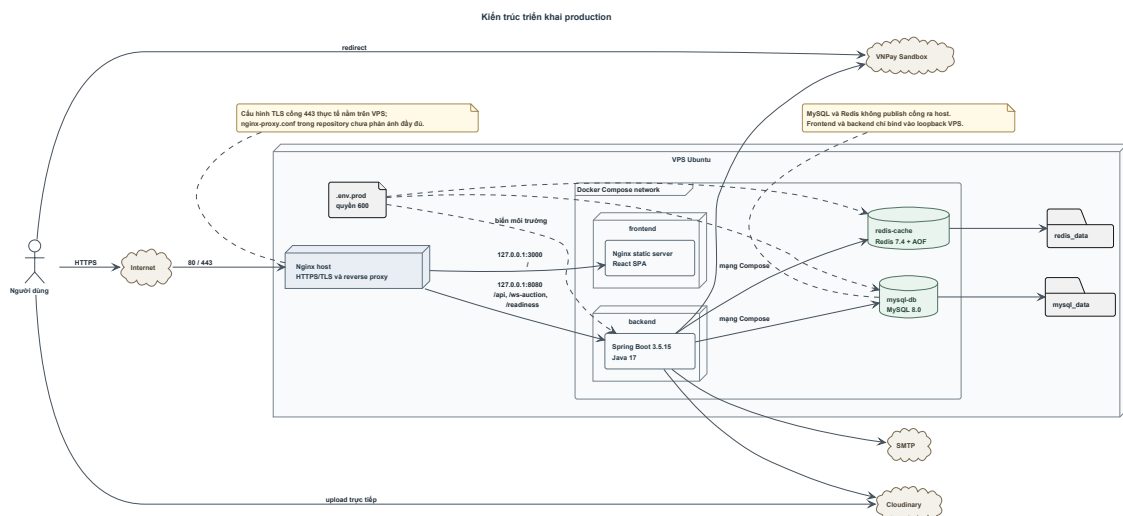
WoodCert Auction được triển khai trên VPS bằng Docker Compose. Nginx, frontend, backend, MySQL và Redis chạy trên VPS; Cloudinary, VNPay Sandbox và SMTP được sử dụng như các dịch vụ bên ngoài. Trang chủ cùng endpoint readiness hiện có thể truy cập qua HTTPS.

0.5.1 Kiến trúc triển khai container hóa

Tệp Compose production định nghĩa bốn service. MySQL 8.0 và Redis 7.4-alpine sử dụng volume riêng; Redis yêu cầu mật khẩu và bật AOF. Backend chạy JAR Spring Boot trên Java 17; frontend phân phát SPA bằng Nginx. MySQL và Redis không publish cổng ra host, chỉ trao đổi trong mạng Compose. Backend và frontend bind cổng vào loopback của VPS để Nginx host chuyển tiếp request.

Backend chỉ khởi động sau khi MySQL và Redis healthy. Health check backend gọi `/actuator/health/readiness`, trong đó readiness phụ thuộc database và Redis. Frontend khởi động sau backend healthy và có health check HTTP riêng. Các điều kiện này giúp sắp xếp thứ tự khởi động, nhưng không thay thế retry trong ứng dụng hoặc giám sát production dài hạn.

Kiến trúc triển khai được thể hiện tại Hình ???. Trình duyệt gửi HTTPS tới Nginx trên VPS. Request `/api/` được chuyển tới backend; `/ws-auction` được nâng cấp WebSocket; readiness phục vụ kiểm tra trạng thái; các đường dẫn còn lại chuyển tới frontend. Nginx trong container frontend chỉ phân phát tệp tĩnh, cache asset có hash và fallback route SPA về `index.html`.



Hình 0.17: Kiến trúc triển khai production

Nguồn: Tổng hợp từ Docker Compose, Dockerfile, Nginx và cấu hình triển khai.

Nginx trên VPS chuyển hướng HTTP 301 sang HTTPS và phục vụ tên miền `woodauction.id.vn` bằng chứng thư Let's Encrypt. Tệp `nginx-proxy.conf` của dự án chỉ định nghĩa server block cổng 80; cấu hình TLS cho cổng 443 nằm trên VPS và không được lưu cùng mã nguồn. Do chưa có tệp cấu hình hoặc log Certbot tương ứng, chưa thể xác định chính xác cách server block 443 được tạo.

0.5.2 Quản lý cấu hình và dữ liệu nhạy cảm

Các giá trị nhạy cảm không được ghi trực tiếp trong mã nguồn hay Docker Compose. Production sử dụng `.env.prod` trên VPS để cung cấp mật khẩu MySQL, Redis, JWT secret, BCrypt hash tài khoản khởi tạo, SMTP, Cloudinary và VNPay. Deployment script yêu cầu file tồn tại và có quyền 600. Tập mẫu `.env.prod.example` chỉ chứa tên biến và giá trị minh họa.

Docker Compose dựng `DB_URL` từ tên database và credential MySQL, đồng thời truyền host nội bộ `mysql-db` và `redis-cache`. Các URL xác thực email, reset password và VNPay return được dựng từ domain production. Refresh cookie bật `Secure` và `SameSite Lax`. Frontend không đọc `.env.prod` khi container chạy; `VITE_API_BASE_URL` và `VITE_WS_BASE_URL` được đưa vào ở bước build image và trở thành nội dung bundle tĩnh.

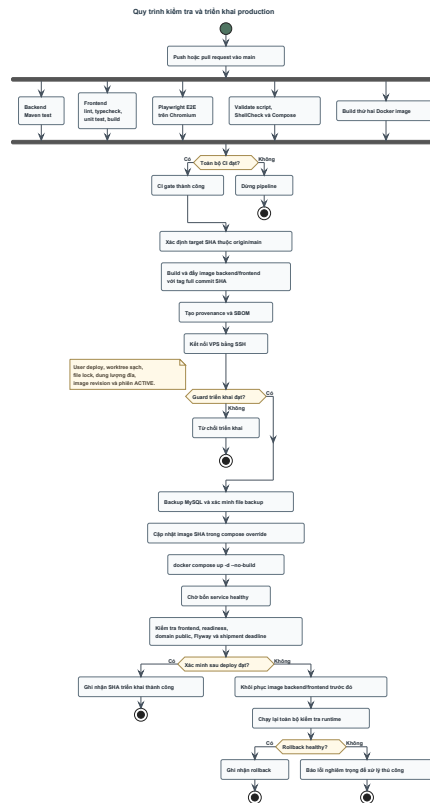
Các secret của GitHub Actions, chẳng hạn SSH private key, chỉ được workflow sử dụng trong runner để kết nối tới VPS; chúng không được đưa vào mã nguồn hoặc image. Các sơ đồ và phần mô tả chỉ nêu nhóm biến cấu hình cùng cách truyền biến, không chứa giá trị thật, private key hay thông tin đăng nhập.

0.5.3 Quy trình tích hợp và triển khai liên tục

Workflow CI chạy khi push hoặc tạo pull request vào nhánh `main`, ngoại trừ thay đổi chỉ nằm trong `thesis/`. Backend job chạy Maven test và xác minh bài kiểm tra wiring production đã được thực thi. Frontend job cài dependency bằng lockfile rồi chạy lint, typecheck, unit test và build. Job riêng cài Chromium và chạy Playwright. Deployment validation kiểm tra cú pháp script, chạy ShellCheck và validate Docker Compose với env file tạm. Hai Docker image cũng được build thử trước khi CI gate chấp nhận pipeline.

Release production được kích hoạt sau khi CI của commit trên `main` thành công hoặc bằng thao tác thủ công có target SHA. Workflow kiểm tra SHA thuộc lịch sử `origin/main`. Backend và frontend được build thành image bất biến trên GHCR, gắn tag bằng full commit SHA và label revision; workflow đồng thời tạo provenance và SBOM.

Quy trình đầy đủ được minh họa tại Hình ???. Trước khi thay container, script yêu cầu chạy bằng user `deploy`, thư mục ứng dụng là đường dẫn tuyệt đối, `worktree` production sạch, còn ít nhất 5 GiB đĩa và không có deployment khác giữ file lock. Image mục tiêu được pull trước và label revision phải khớp target SHA.



Hình 0.18: Quy trình kiểm tra và triển khai production
 Nguồn: Tổng hợp từ GitHub Actions và `scripts/deploy-production.sh`.

Do phiên ACTIVE giữ state runtime trong Redis, deploy mặc định bị chặn nếu còn phiên đang hoạt động. Tùy chọn override chỉ được chấp nhận khi không có bid trong 10 phút gần nhất và không có phiên kết thúc trong 15 phút tới. Đây là biện pháp giảm nguy cơ gián đoạn phòng đặt giá, không phải cơ chế zero-downtime.

Script tạo backup MySQL nhất quán bằng `mysqldump`, nén gzip, kiểm tra file không rỗng và xác minh marker hoàn tất. Sau đó script cập nhật compose override để chọn đúng image SHA và chạy Docker Compose không build lại. Bước xác minh chờ bốn service healthy, gọi readiness và frontend từ loopback lần domain public, kiểm tra không có migration Flyway thất bại và không còn fulfillment chờ giao bị thiếu deadline.

Nếu stack mới không vượt qua xác minh, script đưa backend/frontend về image trước đó và chạy lại toàn bộ kiểm tra runtime. Backup database được tạo để phục vụ khôi phục nếu migration hoặc vận hành phát sinh sự cố, nhưng script không tự động restore database.

0.5.4 Tình trạng hoạt động trên production

Production sử dụng tên miền `woodauction.id.vn` và đã hoạt động ổn định trong quá trình sử dụng. DNS phân giải đúng tên miền, HTTP được chuyển hướng

sang HTTPS, trang chủ trả HTTP 200 và endpoint readiness trả HTTP 200 kèm các header bảo mật của Spring Security.

Các kết quả trên xác nhận WoodCert Auction đã được triển khai production và có thể truy cập tại thời điểm kiểm tra. Dự án chưa có dữ liệu giám sát và benchmark đủ dài để công bố tỷ lệ sẵn sàng, số người dùng đồng thời, thông lượng hoặc khả năng chịu tải.

0.6 Tổng kết chương

Chương ?? đã trình bày cách WoodCert Auction được thiết kế, xây dựng, kiểm thử và triển khai. Backend sử dụng kiến trúc đơn khối phân mô-đun, còn frontend được xây dựng dưới dạng ứng dụng trang đơn. MySQL lưu dữ liệu bền vững; Redis giữ trạng thái runtime của các phiên đấu giá đang hoạt động. Phần thiết kế chi tiết đã làm rõ cấu trúc giao diện, các lớp điều phối nghiệp vụ, trình tự xử lý những use case quan trọng và lược đồ dữ liệu do Flyway quản lý.

Các bộ kiểm thử backend, frontend và đầu cuối đều đạt trong phạm vi kịch bản đã chạy. Hệ thống cũng đã được triển khai và có thể truy cập qua HTTPS. Dự án chưa thực hiện đo tải, kiểm toán hoặc theo dõi dài hạn. Chương ?? tiếp tục phân tích các giải pháp nổi bật về phối hợp Redis–MySQL, bảo vệ ví trước xử lý lặp, và kiểm soát thẩm định bằng dấu vân tay chứng thư.